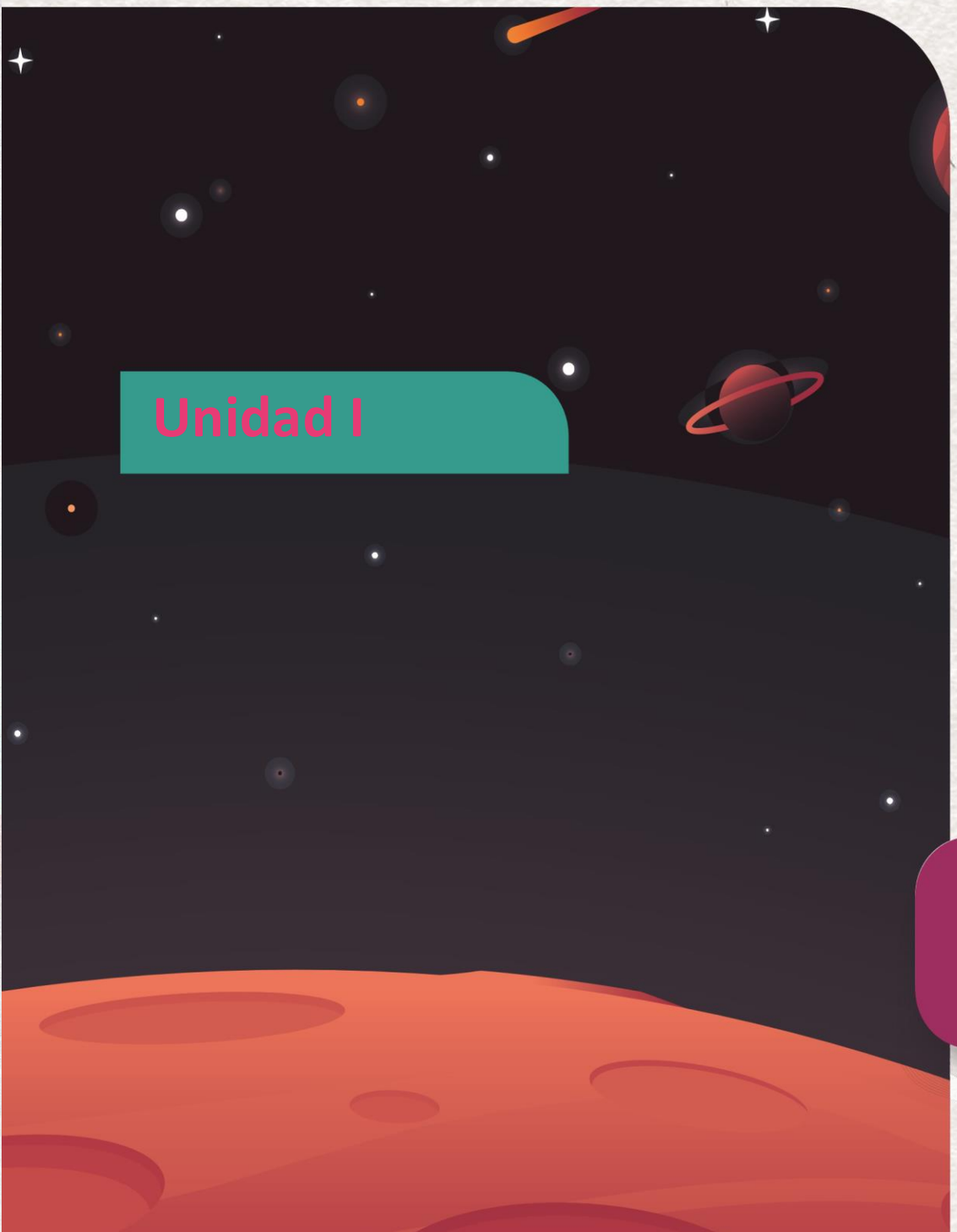
CESDE

CESDE | comfama



Fundamentación técnica en Java (estructuras y modularidad)

- **Colecciones en Java:** List (ArrayList) y operaciones básicas
- **Estructuras clave–valor:** Map (HashMap) y casos de uso
- **Modularidad y reutilización:** métodos (funciones), parámetros, retorno

Fundamentos de POO

Recuerda

Antes de iniciar, debemos tener claro que:

- **Java es Fuertemente Tipado:** No podemos guardar un String donde definimos un Integer.
- **Referencia vs. Valor:** Las colecciones guardan referencias a objetos, no los objetos en sí.
- **Git no es un respaldo, es un historial:** Cada commit debe representar una unidad funcional lógica de código.
-

El Concepto Técnico Static

En Java, cuando declaras una variable como static, le estás diciendo a la Máquina Virtual de Java (JVM) que esa variable **pertenece a la Clase** y no a los objetos individuales que crees a partir de ella.

- **Sin static (De instancia):** Cada vez que creas un objeto (un "restaurante" nuevo), este tendría su propia lista de pendientes vacía. Si tienes 10 restaurantes, tienes 10 listas independientes.
- **Con static (De clase):** Solo existe **una única lista** en toda la memoria del programa para todos los objetos de esa clase. Es una "variable global" dentro del contexto de esa clase

Modularidad Avanzada y Operaciones List

El Contrato de los Métodos

Un método profesional no solo ejecuta; valida. Antes de realizar una operación sobre una List, debemos asegurar que los parámetros sean consistentes para evitar el temido NullPointerException.

Caso de Uso: El "Checklist" de Tareas Automático

Un sistema que permite agregar tareas, pero que las limpia automáticamente si detecta duplicados.

Implementación paso a paso:

1. **Definición:** Creamos una List<String> para el almacenamiento.
2. **Modularización:** Un método exclusivo para la lógica de inserción segura.
3. **Lógica:** Usar el método .contains() antes de .add().

Implementación

```
import java.util.*;

public class SistemaTareas {
    // Estructura dinámica que crece según la necesidad
    static List<String> pendientes = new ArrayList<>();

    public static void main(String[] args) {
        // Simulación de flujo de trabajo
        registrarTarea("Configurar GIT");
        registrarTarea("Crear ArrayList");
        registrarTarea("Configurar GIT"); // Intento de duplicado

        System.out.println("Tareas finales: " + pendientes);
    }

    // Modularidad: Responsabilidad única de validación y registro
    public static void registrarTarea(String tarea) {
        if (tarea != null && !pendientes.contains(tarea)) {
            pendientes.add(tarea);
            System.out.println("Tarea agregada: " + tarea);
        } else {
            System.out.println("Error: Tarea nula o ya existente.");
        }
    }
}
```

Estructuras Map y Casos de Uso Real

La Eficiencia del Hash

Mientras que en una List buscar un elemento implica recorrerla (lento), en un Map la búsqueda es instantánea (O(1)) mediante el uso de llaves únicas.

Caso de Uso: Sistema de Autenticación de Usuarios (Simulado)

En el Back End, los usuarios se buscan por su correo electrónico o ID. Usaremos un Map para simular una base de datos en memoria.

Implementación

```
import java.util.*;

public class AuthSystem {
    // Key: Email (Único), Value: Rol de usuario
    static Map<String, String> baseDatos = new HashMap<>();

    public static void main(String[] args) {
        // Poblado inicial
        baseDatos.put("santiago@cesde.edu", "Admin");
        baseDatos.put("estudiante@cesde.edu", "User");

        // Caso de uso: Login
        verificarAcceso("santiago@ cesde.edu ");
        verificarAcceso("desconocido@web.com");
    }

    public static void verificarAcceso(String email) {
        // Uso de Null Safety al recuperar del mapa
        String rol = baseDatos.get(email);

        if (rol != null) {
            System.out.println("Bienvenido. Su rol es: " + rol);
        } else {
            System.out.println("Acceso denegado: Usuario no encontrado.");
        }
    }
}
```

Integración con GIT

En un entorno real, el código no vive solo en el editor. Debemos aprender a versionar estos ejercicios.

Flujo Práctico en Consola:

1. **Inicialización:** git init (Crea la estación de control).
2. **Estado:** git status (Ver qué "documentos" o cambios hay en la carpeta).
3. **Preparación:** git add . (Poner los cambios en la estación de preparación/staging).
4. **Confirmación:** git commit -m "feat: implementar sistema de login con HashMap"

Conventional Commits

Un commit profesional se estructura así: <prefijo>: <descripción en minúsculas>.

Prefijo	¿Cuándo usarlo?
feat	Cuando añades una nueva funcionalidad al código.
fix	Cuando corriges un error o bug (ej. un NullPointerException).
docs	Cambios únicamente en la documentación (archivos .md, comentarios).
style	Cambios que no afectan la lógica (espacios, formato, punto y coma).
refactor	Código que ni corrige un error ni añade una función, solo se reorganiza.
test	Cuando añades o corriges pruebas unitarias.
chore	Tareas rutinarias, mantenimiento o actualización de dependencias.

Importancia del .gitignore.

Nunca hagas commit de archivos temporales de tu IDE (como la carpeta .idea o .settings). Usa siempre un archivo .gitignore. **¿Cómo?**

Si el archivo es nuevo (Aún no has hecho git add), solo debes escribir el nombre o la ruta en el archivo .gitignore.

1. **Crear el archivo:** En la raíz de tu proyecto, crea un archivo llamado exactamente .gitignore.
2. **Escribir las reglas:** Abre el archivo con cualquier editor de texto y añade los nombres de las carpetas o archivos que deseas ignorar.
3. **Guardar y Commit:** Git ahora ignorará automáticamente esos archivos.

Si el archivo ya fue subido (El error común)

Si ya hiciste commit de la carpeta .idea, no basta con ponerla en el .gitignore; Git ya la está "vigilando". Debes decirle que deje de rastrearla:

1. **Quitar del índice (sin borrar físicamente):** git rm -r --cached .idea/
2. **Añadir al .gitignore:** Escribe .idea/ dentro del archivo.
3. **Hacer commit:** git commit -m "chore: eliminar archivos de configuracion del IDE del rastreo"

Demo Acompañada: "El Gestor de Inventario con Alertas"

Instrucciones: Debes construir un programa que combine todo lo visto:

1. Usa un HashMap<String, Integer> donde la llave es el nombre del producto y el valor es el stock.
2. Crea un método modular actualizarStock(String producto, int cantidad).
3. **El reto de lógica:** Si el stock llega a menos de 5 unidades, el método debe imprimir una alerta de "Stock Crítico" además de actualizar el valor.
4. Al finalizar, realiza un commit.

Ejercicio Independiente: "Sistema de Turnos Bancarios"

Instrucciones:

1. Crea una List para los nombres de los clientes en espera.
2. Crea un método atenderSiguiente() que:
   - Valide si la lista está vacía (Null/Empty Safety).
   - Extraiga el primer cliente (index 0) y lo elimine de la lista.
   - Retorne el nombre del cliente atendido.
3. Sube tu solución a un repositorio de GitHub y comparte el enlace.

¿Debería saber algo más?

Sugerencia Crítica: En Java profesional, no usamos static para todo. Estamos usándolo para facilitar la ejecución en el main, pero la verdadera potencia viene de crear objetos (instancias).

Git - Sugerencia: Existe un sitio web llamado gitignore.io donde escribes "Java" o "IntelliJ" y te genera automáticamente el archivo completo. Es una herramienta profesional que ahorra mucho tiempo.

CESDE | comfama